Computer Project 1: Comparing Analytic and Finite-Difference Numerical solutions to a transient system

Owen Strong
Prof.: Caleb Brooks

March 4th, 2026

Contents

1 Introduction and Background **1**
1.1 Lump Sum Approximation 2
1.2 Finite Difference 4

2 Experimental Procedure **5**

3 Experimental Results **6**

4 Discussion **6**

5 Conclusions **6**

A Numerical Solution Script **8**

Abstract

1 Introduction and Background

Heat transfer is a critical phenomenon to understand in virtually any engineering field, from the cooling of spacecraft in vacuum to the power generation of a nuclear reactor to simply saving money heating a house during winter.

Temperature is a macroscopic phenomenon, describing the average kinetic energy of particles within a medium (as opposed to the bulk motion of the medium). Heat transfer, then, is the thermal energy in transit caused by a spacial difference in temperature. Heat transfer may be understood as a combination of three empirical phenomena: conduction, convection, and radiation, shown in Fig. 1. Of these, conduction and convection will be the primary interest in this report.

Conduction, in which thermal energy diffuses through particles in the medium, follows Fourier's law:

$$\vec{q''}_{cond} = -k\nabla T \tag{1}$$

where $\vec{q''}_{cond}$ is the directional heat flux (i.e. heat transfer per unit area) caused by conduction, ∇T is the temperature gradient, and k is *thermal conductivity*, a material property relating the two.

Convection, in which conduction is combined with bulk motion of a fluid, is described for a surface interface between a material and the fluid, using Newton's law of cooling:

$$\vec{q''}_{conv} = h(T_f - T_s) \tag{2}$$

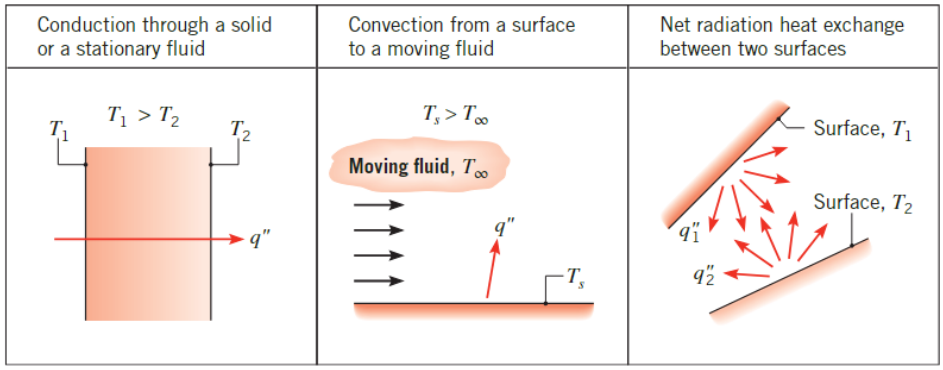

Figure 1: Conduction, convection, and radiation heat transfer modes. Figure sourced from *Fundamentals of Heat and Mass Transfer*.[1]

Where T_f is the temperature of the convective fluid, T_s the temperature at the surface, and $\vec{q''}_{cond}$ is the directional heat flux resulting from this convection.

Fourier's law of conduction can be related to a control volume to derive the General Heat Diffusion Equation (GHDE) for a particular location within a material[1]:

$$\rho c_p \frac{\partial T}{\partial t} = -\nabla k \nabla T + q''' \tag{3}$$

where ρ represents mass density and c_p the heat capacity of the material, and q''' the volumetric heat generation at that location. For a material with uniform properties, this is typically simplified to:

$$\frac{1}{\alpha}\frac{\partial T}{\partial t} = \frac{q'''}{k} - \nabla^2 T \tag{4}$$

where α is the thermal diffusivity of the material:

$$\alpha = \frac{k}{\rho c_p} \tag{5}$$

This equation, in either form, can be solved analytically. However, doing so in cases considering anything more than one dimension (including considering time) is non-trivial, and quickly devolves into tedious calculus including summations of eigenfunctions.[1]

For a transient one-dimensional problem (one with time dependence), there are two immediate techniques to avoid this analysis: the lumped sum approximation, and a numerical simulation.

In this report, an analytic solution using the lumped sum approximation will be compared to a finite difference numerical solution, for a problem involving a uniform sphere subjected to convection in a cooling fluid.

1.1 Lump Sum Approximation

Consider a uniform sphere which is highly conductive, subjected to convective cooling by a fluid surrounding it. One could solve for the temperature distribution within the sphere

over time using Equation 4, but if the sphere is highly conductive (high k) compared to the convective ability of the fluid and surface (low h), then one can instead assume a uniform temperature $\bar{T}$ over the surface. Applying a control volume around the sphere, uniform over its volume V, with a surface area A in contact with the fluid:

$$\rho c_p V \frac{d\bar{T}(t)}{dt} = -hA(\bar{T}(t) - T_f) + q''' \tag{6}$$

This equation, which now only considers one variable, can be solved by changing perspective to the relative temperature $\bar{\theta} = \bar{T} - T_f$ (assuming heat generation $q''' = 0$):

$$\begin{aligned}
\rho c_p V \frac{d\bar{\theta}}{dt} &= -hA(\bar{\theta}) + q''' \\
\frac{d\bar{\theta}}{dt} &= -\frac{hA}{\rho c_p V}\bar{\theta} \\
\frac{1}{\bar{\theta}} d\bar{\theta} &= -\frac{hA}{\rho c_p V} dt \\
\ln(\bar{\theta}) &= -\frac{hA}{\rho c_p V} t + C \\
\rightarrow \tau &= \frac{hA}{\rho c_p V} \\
\ln(\bar{\theta}) &= -\frac{t}{\tau} + C \\
\bar{\theta} &= Ce^{-\frac{t}{\tau}} \\
\rightarrow \bar{\theta}(t = 0) &= \bar{\theta}_0 \\
\bar{\theta}_0 &= Ce^{-\frac{0}{\tau}} = C \\
\bar{\theta} &= \bar{\theta}_0 e^{-\frac{t}{\tau}}
\end{aligned} \tag{7}$$

If one component out of the material properties contained in the time constant τ, the initial temperature difference $\bar{\theta}_0$, the time t, or the temperature difference $\bar{\theta}$ at that time, this equation can be used to solve for the missing value of otherwise known system, such as the convection coefficient necessary in a fluid for a given conductive sphere to go from an initial temperature to a known final temperature within that fluid after a known time.

To consider whether it is appropriate to use the lumped sum approximation, the Biot number, a dimensionless quantity relating thermal convection to conduction, should be appropriately low:

$$B_i = \frac{hL}{k} \leq 0.1 \tag{8}$$

where L is the characteristic length of the system in question, in this case the diameter of the sphere, and k and h the conductive material's conductivity and the fluid-material interface's convection coefficient, respectively.

1.2 Finite Difference

If, instead, one discretizes Equation 4 (once again assuming no heat generation) over finite differences in time ($t = p\Delta t$) and the radius of the sphere ($r = n\Delta r$):

$$\frac{1}{\alpha}\frac{\partial T}{\partial t} = \frac{q'''}{k} - \nabla^2 T$$

$$\frac{1}{\alpha}\frac{\partial \bar{\theta}}{\partial t} = -\frac{1}{r^2}\frac{d}{dr}\left(r^2\frac{d\bar{\theta}}{dr}\right)$$

$$\frac{1}{\alpha}\frac{\partial \bar{\theta}}{\partial t} = -\frac{1}{r^2}\frac{d}{dr}\left(n^2\Delta r^2\frac{T_{n+\frac{1}{2}} - T_{n-\frac{1}{2}}}{\Delta r}\right)$$

$$\frac{1}{\alpha}\frac{\partial \bar{\theta}}{\partial t} = -\frac{1}{r^2}\frac{d}{dr}\left(n^2\Delta r(T_{n+\frac{1}{2}} - T_{n-\frac{1}{2}})\right)$$

$$\frac{\partial \bar{\theta}}{\partial t} = \frac{\alpha}{\Delta r^2}\frac{(n+\frac{1}{2})^2[T_{n+1} - T_n] - (n-\frac{1}{2})^2[T_n - T_{n-1}]}{n^2}$$

discretizing over time such that the gradient is evaluated at the previous timestep p (*"explicit"* finite difference) and introducing the differential system's Fourier number F_o:

$$\frac{\partial \bar{\theta}}{\partial t} = \frac{\alpha}{\Delta r^2}\frac{(n+\frac{1}{2})^2[T_{n+1} - T_n] - (n-\frac{1}{2})^2[T_n - T_{n-1}]}{n^2}$$

$$\frac{T_n^{p+1} - T_n^p}{\Delta t} = \frac{\alpha}{\Delta r^2}\frac{(n+\frac{1}{2})^2[T_{n+1}^p - T_n^p] - (n-\frac{1}{2})^2[T_n^p - T_{n-1}^p]}{n^2}$$

$$\frac{T_n^{p+1} - T_n^p}{=}\frac{\alpha\Delta t}{\Delta r^2}\frac{(n+\frac{1}{2})^2[T_{n+1}^p - T_n^p] - (n-\frac{1}{2})^2[T_n^p - T_{n-1}^p]}{n^2}$$

$$\rightarrow F_o = \frac{\alpha\Delta t}{\Delta r^2}$$

$$T_n^{p+1} - T_n^p = F_o\frac{(n+\frac{1}{2})^2[T_{n+1}^p - T_n^p] - (n-\frac{1}{2})^2[T_n^p - T_{n-1}^p]}{n^2}$$

$$T_n^{p+1} = T_n^p + F_o\frac{(n+\frac{1}{2})^2[T_{n+1}^p - T_n^p] - (n-\frac{1}{2})^2[T_n^p - T_{n-1}^p]}{n^2} \tag{9}$$

By selecting offset indices of $n \in \{\frac{1}{2}, \frac{3}{2}, \frac{5}{2}...\}$, one may dictate a symmetric boundary condition where no heat passes through the center of the sphere, making the node $n = \frac{1}{2}$ only dependent on its outer neighbor $\frac{3}{2}$:

$$T_n^{p+1} = T_n^p + F_o\frac{(n+\frac{1}{2})^2[T_{n+1}^p - T_n^p] - (n-\frac{1}{2})^2[T_n^p - T_{n-1}^p]}{n^2}$$

$$\rightarrow n = \frac{1}{2}$$

$$T_n^{p+1} = T_n^p + F_o\frac{(1)^2[T_{n+1}^p - T_n^p] - (0)^2[T_n^p - T_{n-1}^p]}{\frac{1}{2}^2}$$

$$T_{\frac{1}{2}}^{p+1} = T_{\frac{1}{2}}^p + F_o\frac{T_{\frac{3}{2}}^p - T_{\frac{1}{2}}^p}{\frac{1}{2}^2}$$

And, for the outermost node n_s placed exactly at the surface of the sphere meeting the convective fluid, applying a control volume meeting the previous node at the internal conducting control surface $n_s - \frac{1}{2}$:

$$\frac{V_{n-\frac{1}{2},n}\rho c_p}{\delta t}(T_n^{p+1} - T_n^p) = kA_{n-\frac{1}{2}}\frac{T_{n-1}^p - T_n^p}{\Delta r} + hA_n(T_f - T_n^p)$$

$$\rightarrow V_{n-\frac{1}{2},n} = \frac{4}{3}\pi(n^3 - (n - \frac{1}{2})^3)$$

$$\rightarrow A_n = 4\pi n^2$$

$$T_n^{p+1} - T_n^p = 3F_o\frac{(n-\frac{1}{2})^2}{n^3 - (n-\frac{1}{2})^3}(T_{n-1}^p - T_n^p) + 3F_oB_i\frac{n^2}{n^3 - (n-\frac{1}{2})^3}(T_f - T_n^p)$$

or more concisely, by establishing dimensionless internal and edge geometric attenuation coefficients $\Gamma_i = \frac{(n_s-\frac{1}{2})^2}{n_s^3-(n_s-\frac{1}{2})^3}$, $\Gamma_e = \frac{n_s^2}{n_s^3-(n_s-\frac{1}{2})^3}$

$$T_{n_s}^{p+1} = (1 - 3F_o\Gamma_i - 3F_oB_i\Gamma_e)T_{n_s}^p + 3F_o\Gamma_iT_{n_s-1}^p + 3F_oB_i\Gamma_eT_f \tag{10}$$

As ensuring stability requires a positive coefficient of the current temperature T_n^p, this yields a strict stability criterion:

$$\frac{1}{3} \geq F_o(\Gamma_i + B_i\Gamma_e) \tag{11}$$

Since in Equation 11, the characteristic length and time in F_o and B_i correspond to the differential length Δr and time Δt, B_i (only dependent on geometry) can be used to solve for the required F_o,

$$F_o \leq \frac{1}{3(\Gamma_i + B_i\Gamma_e)} \tag{12}$$

then solved for the maximum stable timestep Δt:

$$\frac{\alpha\Delta t}{\Delta r^2} \leq \frac{1}{3(\Gamma_i + B_i\Gamma_e)}$$

$$\boxed{\Delta t \leq \frac{\Delta r^2}{3\alpha(\Gamma_i + B_i\Gamma_e)}}$$

2 Experimental Procedure

A `Python` script was constructed, implementing Equations 9, 10, and 11 to solve for the temperature distribution within a pure copper sphere (Using tabulated material propertiy data[1]). This script, as well as accompanying sample input and output files, are included in Appendix A.

Using the numerical solution in Equation 6

3 Experimental Results

4 Discussion

5 Conclusions

References

[1] T. L. Bergman and A. S. Lavine, *Fundamentals of heat and mass transfer*, Eighth edition. Hoboken, NJ: John Wiley & Sons, 2017, 1 p., ISBN: 978-1-119-32042-5 978-1-119-33767-6.

A Numerical Solution Script

The full `Python` script used to evaluate the explicit finite difference scheme is included below:

```
import json
import os.path

INPUT_PATH="input.json"
BASE_OUTPUT_PATH="output/output"
input = json.load(open(INPUT_PATH))

# Copper Material Properties
rho = 8933 # kg / m^3
c_p = 385 # J / kg*K
if "k_Wm-1K-1" in input:
    k = input["k_Wm-1K-1"] # W / m*K, default of 401
else:
    k = 401
alph = k / (rho*c_p) # (m^2 / s)

# Fluid properties
h = input["h_Wm-2K-1"] # W/m^2*K
T_INF = input["T_INF_C"] # C

# Experiment Setup
R = input["R_m"] # m
N_NODES = input["N_NODES"]
dr = R / (N_NODES - 0.5) # m, chosen so that final node (N-1) + 0.5 corresponds to the r
T_0 = input["T_0_C"] # C
t_final = input["total_time_s"] # s

# Secondary setup (necessary dt)
SAFETY = input["SAFETY"] # Use smaller Fo than necessary
Bi = h*dr/k # Biot number, dimless
n_edge = N_NODES-0.5 # n of the last node, corresponds to radius of surface
n_inge = n_edge-0.5 # n of the border with second to last node (inside edge)
Fo = SAFETY * (1 / (3 * (n_inge**2/(n_edge**3 - n_inge**3)  +  Bi*n_edge**2/(n_edge**3 -
# print(f"Biot: {Bi}, Fo: {Fo}")
dt = Fo * dr**2 / alph
if "dt" in input and not input["dt"]=="auto":
    dt = input["dt"]
    Fo = dt*alph / (dr**2)
print(f"dt: {dt} s")

T_vals:list = [T_0 for i in range(N_NODES)] #
```

```
n_vals:list = [i+0.5 for i in range(N_NODES)] # The actual value n of each node (where r


t_elapsed = 0
t_steps = []
T_vals_sets = [] #List of every T distribution at each timestep.
p = 0
num_timesteps = t_final / dt
num_prints = min(input["num_t_output"], num_timesteps)
print_freq = int(num_timesteps / num_prints)

def push():
    print(f"T({t_elapsed:2.3f}s) = {[f"{T:.3f}" for T in T_vals]}C")
    T_vals_sets.append(T_vals)
    t_steps.append(t_elapsed)

push()

while (t_elapsed < t_final):
    T_vals_next = [0 for i in range(N_NODES)] # Filler list

    if N_NODES > 1:
        # Center node
        T_n0 = T_vals[0]
        T_n1 = T_vals[1]
        n0 = n_vals[0]
        nh = n0 + 0.5
        T_vals_next[0] = T_n0 + Fo * (nh**2 / n0**2) * (T_n1-T_n0)

        for i in range(1, N_NODES-1): # Interior nodes, not special cases
            n = n_vals[i]
            T_n = T_vals[i]
            T_i = T_vals[i-1]
            T_o = T_vals[i+1]

            T_vals_next[i] = T_n + Fo * ((n+0.5)**2*(T_o-T_n) - (n-0.5)**2*(T_n-T_i)) /
            # print(f"new T_{i}: {T_vals_next[i]}")

        # Exterior node
        T_nf = T_vals[-1]
        T_vals_next[-1] = T_nf + 3*Fo * ((n_inge**2/(n_edge**3 - n_inge**3))*(T_vals[-2]
    else:
        T_nf = T_vals[0]
        T_vals_next[0] = T_nf + 3*Fo * (Bi*n_edge**2/(n_edge**3 - n_inge**3))*(T_INF-T_n
```

```
    t_elapsed += dt
    T_vals = T_vals_next
    p+=1

    if (p % print_freq == 0 or p < 10):
        push()
push()

output = {
    "input":input,
    "Biot (dr)":Bi,
    "Biot (D)":h*(2*R)/k,
    "t_steps":t_steps,
    "T_vals":T_vals_sets,
    "dt":dt
}

next_idx = 0
output_path = BASE_OUTPUT_PATH + ".json"
while os.path.isfile(output_path):
    output_path = BASE_OUTPUT_PATH + f"_{next_idx}.json"
    next_idx+=1
json.dump(output, open(output_path, "w"), indent=4)
```

The script accepts an input such as the example `input.json`:

```
{
    "h_Wm-2K-1": 49.95605733,

    "N_NODES": 10,
    "dt": "auto",
    "T_INF_C": 25,
    "T_0_C": 84,
    "total_time_s": 1992.525237,
    "R_m": 0.01,
    "k_Wm-1K-1": 401,
    "num_t_output": 50,
    "SAFETY": 0.75
}
```

And produces a corresponding output such as `output.json`:

```
{
    "input": {
```

```
    "k_Wm-1K-1": 401,
    "T_INF_C": 25,
    "T_0_C": 84,
    "total_time_s": 1992.525237,
    "R_m": 0.01,
    "N_NODES": 10,
    "h_Wm-2K-1": 49.95605733,
    "dt": "auto",
    "num_t_output": 50,
    "SAFETY": 0.75
},
"Biot (dr)": 0.00013113547008793803,
"Biot (D)": 0.002491573931670823,
"dt": 0.0037647702348469264,
"t_steps": [
    0,
    0.0037647702348469264,
    ...
    1992.527235387602
],
"T_vals": [
    [
        84,
        84,
        84,
        84,
        84,
        84,
        84,
        84,
        84,
        84
    ],
    [
        84.0,
        84.0,
        84.0,
        84.0,
        84.0,
        84.0,
        84.0,
        84.0,
        84.0,
        83.99353554087601
    ],
```

```
        ...
        [
            25.009797880330684,
            25.009797778653294,
            25.009797524462186,
            25.009797129061415,
            25.009796595285344,
            25.00979592427711,
            25.00979511661838,
            25.009794172652125,
            25.009793092603644,
            25.009791876634377
        ]
    ]
}
```

where ‘‘...’’ denotes values truncated for brevity.